E-commerce Infrastructure as Code

Final Project Summary | Terraform on AWS | Merter Akinci

Status: Completed (Learning Phase 1-3) | All AWS resources have been destroyed to avoid ongoing costs. Infrastructure can be fully recreated at any time from the Terraform code in this repository.

1. Project Overview

This project was a hands-on learning exercise in Infrastructure as Code (IaC) and AWS cloud security fundamentals, using Terraform to provision the networking and data layer of a hypothetical e-commerce platform. The project intentionally stopped before deploying the application itself, as the learning path continues into container orchestration (Kubernetes) in a separate, dedicated repository.

Repository: **github.com/merterakinci/ecommerce-infra-as-code**

2. What Was Built

Layer	Resources	Purpose
Storage	S3 bucket (with versioning + lifecycle policy)	Object storage, protected against accidental overwrite/deletion
Networking	VPC, 2 public/private subnets, Internet Gateway, Route Table	Isolated network, segregating public-facing and internal re
Access Control	2 Security Groups (web-server-sg, database-sg)	least-privilege firewall rules; database only reachable from web t
Database	RDS PostgreSQL instance (private, non-public)	Managed relational database, deployed in private subnets only

3. Repository Structure

```
.
■■■ main.tf               # Provider config + S3 bucket, versioning, lifecycle policy
■■■ network.tf            # VPC, subnets, Internet Gateway, route table
■■■ security_groups.tf    # Web server and database security groups
■■■ database.tf           # RDS PostgreSQL instance + DB subnet group
■■■ variables.tf          # Input variables (region, environment, credentials)
■■■ outputs.tf            # Output values (bucket name, ARN)
■■■ .gitignore            # Excludes state files and secrets
■■■ README.md             # Documentation and roadmap
```

4. Skills Demonstrated

- Infrastructure as Code fundamentals: provider configuration, resource definitions, variables, outputs, state management
- AWS core services: VPC networking, IAM, S3, RDS, Security Groups

- Cloud security principles: least privilege access, network segmentation (public/private subnets), identity-based security group rules, credential hygiene
- Incident response in practice: identified and remediated an exposed AWS access key during setup (deactivated, deleted, rotated)
- Git/GitHub workflow: repository setup, structured commits, .gitignore for sensitive files, interactive rebase, Personal Access Token authentication
- Cost awareness: AWS Free Tier usage, budget alerts, Cost Anomaly Detection, and full teardown via terraform destroy to avoid ongoing charges

5. Why the Project Stopped Here

The original roadmap considered deploying the application on AWS EC2 or ECS. However, since the learning goal shifted toward Kubernetes (targeting the CKA and CKS certifications), continuing the application deployment on this AWS infrastructure would have introduced unnecessary cost (EC2/EKS) without directly supporting that certification path. All AWS resources were destroyed via `terraform destroy` to eliminate ongoing costs, while the Terraform code remains in version control and can be re-applied at any time.

6. Next Steps

Kubernetes learning continues in a separate repository (**k8s-labs**), using a local Minikube cluster to avoid cloud costs while preparing for the CKA/CKS certification path. The AWS/Terraform skills demonstrated here remain available to revisit for future cloud-native projects, including a potential future integration with a managed Kubernetes service (e.g. AWS EKS).

Part of a self-directed learning path in Cloud Security and DevSecOps, building on a background in SIEM/SOC operations (CompTIA Security+, ISC2 CC).